

6. LLM Advance Quantization Introduction

Term Meaning & Clarification

- FP** Floating Point ✓
- INT** Integer (e.g., INT8, INT4 used for quantization)
- PTQ** Post-Training Quantization — quantize a model **after full training** without re-training
- QAT** Quantization-Aware Training — simulate quantization during training to **preserve accuracy**
- GPTQ** Originally stands for "**Gradient Post-Training Quantization**". It is a **specific PTQ technique** designed for transformers. People confuse it with "Generative Pretrained Transformer Quantization", but that is incorrect.
- AWQ** Activation-aware Weight Quantization — focuses on **scaling weights w.r.t activations** for better accuracy in INT4/INT8
- GGM** Georgi Gerganov's Machine Learning library for inference — supports CPU/GPU/backends
- L**
- GGUF** Gerganov's General Unified Format — successor to GGML binary formats. Used to store **quantized models** (e.g., LLaMA, Mistral) for use in tools like llama.cpp.
- ! Some people say it stands for "GPT-Generated Unified Format" but that's a **backronym**, not official.

Extra Notes (for depth):

- GPTQ = *Post-Training Quantization specifically for Transformers*
- GGUF = *Efficient model file format used by llama.cpp ecosystem*
- GGML = *Inference engine that consumes GGUF or older GGML format for CPU/low-resource inference*

One-Line Summary (Crisp & Final)

- PTQ, QAT** = *Quantization techniques* (how quantization is done).
- GPTQ, AWQ** = *Quantization algorithms/libraries* (enhanced PTQ methods for LLMs).
- GGUF** = *Model file format* (used for saving/exporting quantized models).
- GGML** = *Inference runtime* (engine that runs quantized models like GGUF).

PTQ → GPTQ → GGUF → llama.cpp or GGML

Refined Example Flow (End-to-End):

- You have a **LLaMA-3 8B model** with **FP16 weights**. PTQ
- You perform **quantization using GPTQ** (which is a type of **Post-Training Quantization**).
- You **export the quantized model in GGUF format** (which is compatible with llama.cpp).
- Finally, you **run the model using llama.cpp or a GGML-based engine** — on **CPU or Mac**, no GPU required.

GGML

- GGML = **low-level C library inference runtime**.
- Optimized for **CPU inference** (also has support for Metal on macOS, and some GPU offloading).
- Used as the **core backend** in projects like llama.cpp, whisper.cpp, etc.
- Files like ggml-model-q4_0.bin are indeed **quantized model weight files** stored in GGML's custom format.

llama.cpp

- llama.cpp = **C++ implementation** built on top of GGML/GGUF for running LLaMA-family models locally.
- Provides **CLI + C API + Python bindings** (via llama-cpp-python).
- Supports **quantized GGML/GGUF model formats** → efficient inference on CPU/Mac/low-VRAM GPU.
- "High-performance C++ wrapper" is almost right — actually it's more of a **full C++ project** that *uses GGML/GGUF as backend*. Not just a wrapper.

Quantization Libraries:

- GPTQ (AutoGPTQ)** — Fastest for 4-bit quantization (Hugging Face compatible).
- AWQ python library** — 4-bit quantization with activation-aware optimization (better accuracy).
- bitsandbytes** — Enables INT8 / 4-bit loading directly in Hugging Face models.

Model File Format:

- GGUF** — Efficient file format for storing quantized models (used with GGML or llama.cpp).

Inference Runtimes:

- GGML** — C-based lightweight inference engine (runs quantized models on CPU/Mac).
- llama.cpp** — Wrapper around GGML with CLI + API interface for easy use.

GPU/Server Deployment:

- TensorRT-LLM, vLLM, ONNX Runtime** — High-throughput GPU inference on server setups.

Quantization Workflow Sequence

- You perform **quantization** — using any of the following methods:

- PTQ (Post Training Quantization)
- QAT (Quantization Aware Training)
- GPTQ (Gradient-based PTQ for Transformers)
- AWQ (Activation-aware Weight Quantization)

2. Then you save the quantized model in a specific format, such as:
 - **GGUF** (popular for the LLaMA + llama.cpp ecosystem)
 - **safetensors** (secure tensor format used in Hugging Face Transformers)
 - **ONNX** (used for inference with ONNX Runtime)
 - **TensorRT engine** (for NVIDIA-optimized inference)
3. And finally, you need an engine to run the saved format, such as:
 - **llama.cpp** (for GGUF models) **GGML** (older format, now replaced by GGUF)
 - **vLLM** (used with safetensors for high-throughput inference)
 - **TensorRT-LLM** (optimized for INT4/INT8 inference on NVIDIA GPUs)

Tool / Format	Short Refined Explanation
GPTQ	Post-Training Quantization method (weight-only) for LLMs. Hugging Face + AutoGPTQ integration allows fast 4-bit inference on GPUs. Good balance of speed & accuracy.
AWQ	Activation-aware Weight Quantization . Focuses on minimizing accuracy drop in 4-bit models by preserving important activation patterns. Often slightly better quality than GPTQ in same bit-width.
bitsandbytes	Popular Hugging Face-compatible library for 8-bit & 4-bit loading (LLM.int8(), nf4). Very easy for quick memory savings without retraining.
GGML	C-based inference runtime for quantized models. Optimized for CPU and lightweight GPUs. Backbone for llama.cpp(ggml-model-q4_0.bin)
llama.cpp	High-performance C++ wrapper for GGML/GGUF models. Offers easy CLI tools + C/C++/Python APIs for running LLMs locally without heavy dependencies.
GGUF	Quantized model file format used in llama.cpp ecosystem. Stores weights efficiently in various bit-widths for fast CPU/Mac/low-VRAM GPU inference. Successor to GGML's .bin format.
TensorRT-LLM	NVIDIA library for optimized GPU inference with quantized or FP16/BF16 models. Great for production servers.
vLLM	High-throughput LLM serving engine. Optimized for batching, paged attention, and multi-GPU scalability.
ONNX Runtime	Cross-platform runtime for models exported in ONNX format . Supports CPU, GPU, TensorRT execution with quantization.

Can We Do PTQ/QAT on LLMs?

Yes — **Post-Training Quantization (PTQ)** and **Quantization-Aware Training (QAT)** are both applicable to LLMs, but the process is significantly different from small-scale models like CNNs or MLPs.

PTQ on LLMs

In MLPs (simple case):

$$\text{scale} = (\text{rmax} - \text{rmin}) / (\text{qmax} - \text{qmin})$$

$$\text{zero_point} = \text{round}(-\text{rmin} / \text{scale} + \text{qmin})$$

Done on small tensors (e.g., linear layers)

Can use torch.quantization directly

In LLMs:

- Same core formula, but with **huge tensors**.
- Quantization is done:
 - **Layer-wise** or **per-channel**
 - On weights like **Q/K/V, Feed Forward (FFN)**, etc.
- Requires a **small calibration dataset** (~50–200 samples).
- Very sensitive to precision loss

Tools for PTQ on LLMs:

- **GPTQ** (Gradient-aware PTQ)
- **AWQ** (Activation-aware Weight Quantization)
- **AutoGPTQ** (wrapper around GPTQ + Hugging Face)
- **bitsandbytes** (8-bit & 4-bit loaders)

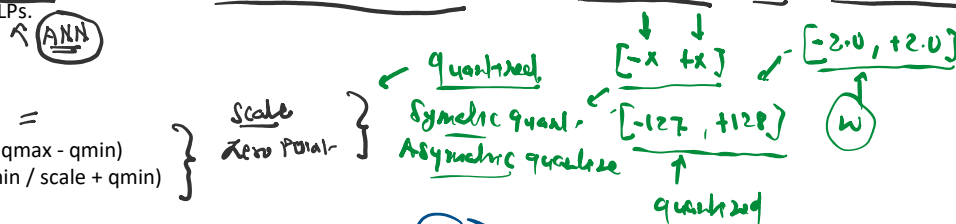
QAT on LLMs

In MLPs (simple case):

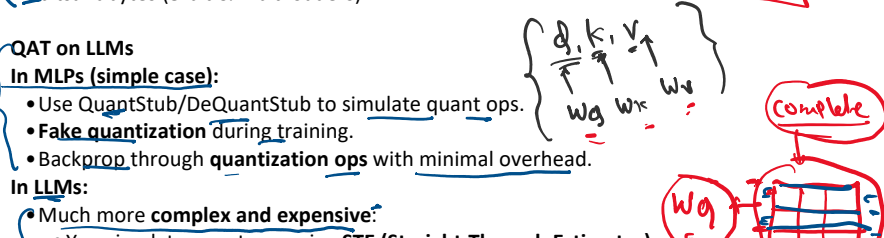
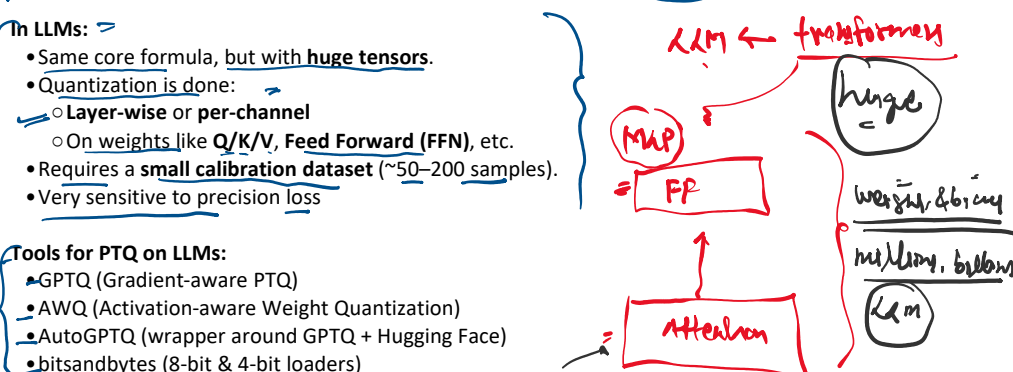
- Use **QuantStub/DeQuantStub** to simulate quant ops.
- **Fake quantization** during training.
- Backprop through **quantization ops** with **minimal overhead**.

In LLMs:

- Much more **complex and expensive**:



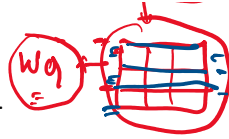
Library ← module (PyTorch)



- Backprop through quantization ops with minimal overhead.

In LLMs:

- Much more complex and expensive:
 - You simulate quant ops using STE (Straight-Through Estimator).
 - Requires full or partial retraining of the model.
 - Requires multi-GPU or TPU setups (very compute-heavy).



transformer

Tools for QAT on LLMs:

- torchao (PyTorch QAT backend)
- Intel Neural Compressor
- NVIDIA TensorRT-LLM (QAT support)
- OpenVINO (QAT & PTQ optimizations)
- HQQ (High-Quality Quantization + QAT)
- SmoothQuant (can be combined with QAT)

Quantized aware training

LLM expensive

MAP, CNN

Component	Notes
◦ <u>Activation quantization</u>	Challenging due to softmax, layernorm, attention
◦ <u>Weight quantization</u>	Done on Q/K/V, FFN, etc., often with per-channel granularity
◦ <u>Techniques</u>	Dynamic, static, and fused quantization techniques are often combined

Aspect	MLPs	LLMs (e.g., GPT, LLaMA)
PTQ	Easy (<u>torch.quantization</u>)	Layer-wise, needs calibration, uses GPTQ/AWQ
QAT	Light training needed	Heavy retraining with STE, uses advanced toolkits
Activation Handling	Simple	Complex (softmax, attention layers)
Tools	Built-in torch	GPTQ, AWQ, bitsandbytes, torchao, TensorRT-LLM
Hardware Need	Single GPU	Multi-GPU/TPU for QAT

Per-Tensor Quantization

Definition:

- The entire tensor (e.g., full weight matrix) shares a single scale and zero-point.

Use Case:

- Simpler and faster.
- Often used for activations.
- But less accurate, especially if different channels have different value ranges.

Example:

Imagine a weight tensor of shape (4, 3):

[0.1, 0.2, 0.3],	GPU tensor 4x3 quantized 1 scale 1 zero-point
[1.5, 2.0, 1.8],	
[0.05, 0.07, 0.08],	
[3.0, 2.5, 3.2]	

MAP, CNN, RNN

In per-tensor, we calculate 1 scale + 1 zero-point for all values above.

Even though the ranges vary across rows, they all share the same quantization formula.

2. Per-Channel Quantization

Definition:

- Each channel (e.g., each row in weight matrix) gets its own scale and zero-point.

Use Case:

- Used mostly for weights in CNNs, LLMs, etc.
- More accurate.
- Especially useful when each channel has a different range of values.

Example:

Using same matrix:

[0.1, 0.2, 0.3],	# Channel 1 → small range → scale1
[1.5, 2.0, 1.8],	# Channel 2 → medium range → scale2
[0.05, 0.07, 0.08],	# Channel 3 → tiny range → scale3
[3.0, 2.5, 3.2]	# Channel 4 → large range → scale4

In per-channel, we compute 4 different scales and 4 zero-points (one for each row).

This allows fine-tuned quantization and reduces loss in accuracy.

Feature	Per-Tensor	Per-Channel
Granularity	Single scale/zero-point	One scale/zero-point per channel

This allows fine-tuned quantization and **reduces loss in accuracy**.

Feature	Per-Tensor	Per-Channel
Granularity	Single scale/zero-point	One scale/zero-point per channel
Speed	Faster (simpler)	Slightly slower
Accuracy	Lower	Higher
Use Case	Activations	Weights (especially in <u>LLMs</u> , <u>CNNs</u>)
Preferred for	Small models or quick inference	High accuracy + transformer models

☑ In Practice (LLMs & CNNs):

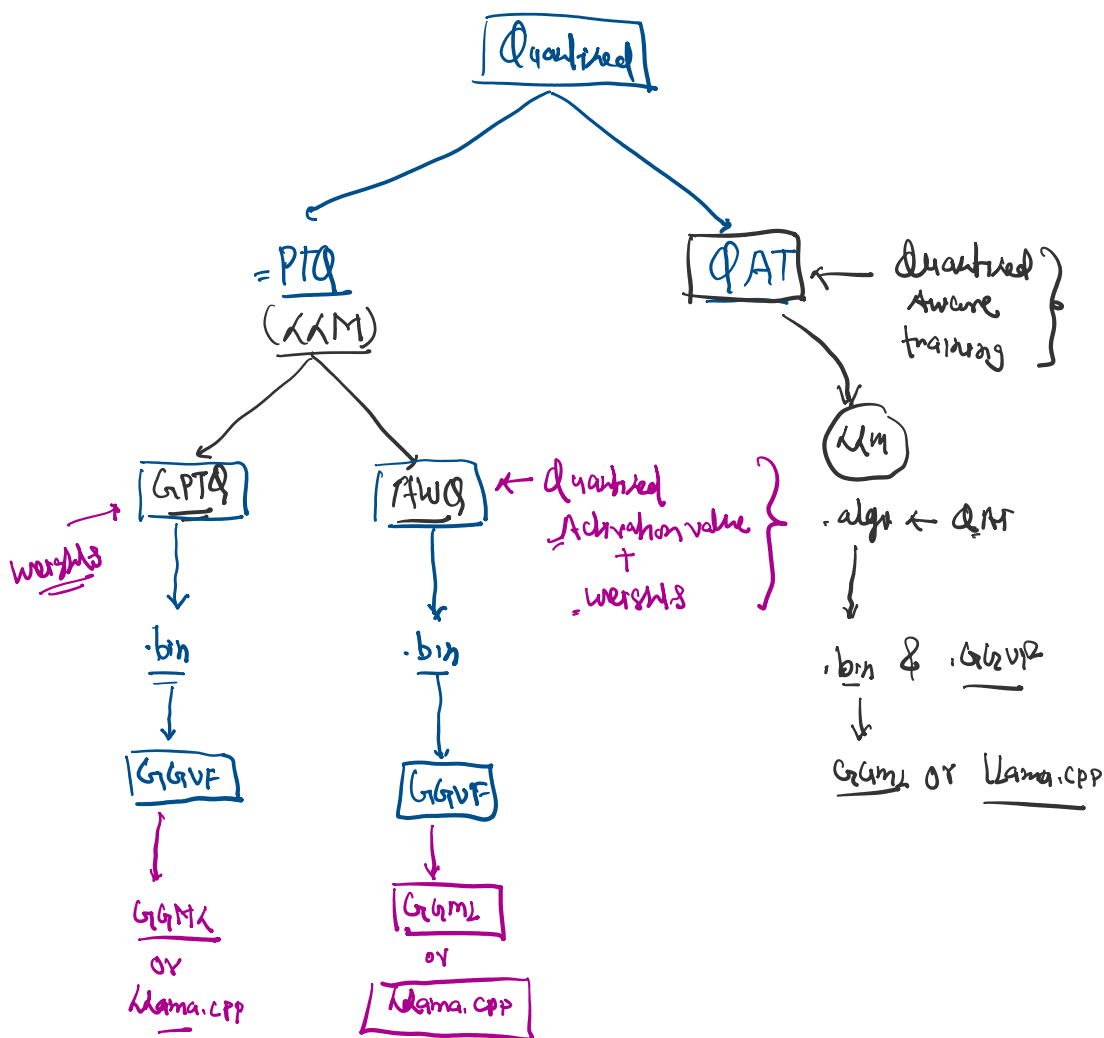
- LLMs mostly use:
Symmetric + Per-Channel Quantization for weights (efficient & accurate).
- CNNs/activations might use:
Asymmetric + Per-Tensor Quantization for dynamic range handling.

In Transformers (like LLaMA, BERT):

- Query, Key, Value, and FFN weights are **quantized per-channel** (row-wise) to preserve model behavior.
- This is usually **symmetric + per-channel**.

Summary

- Per-tensor = simple, fast, but may lose accuracy if value distribution varies.
- Per-channel = precise, handles varying ranges well — best for large models and weights.



- **GPTQ (Gradient Post-Training Quantization)** is an **advanced Post-Training Quantization (PTQ)** method specifically designed for **Large Language Models (LLMs)** like LLaMA, GPT, Mistral, Falcon, etc.
- It quantizes model **weights to low precision (e.g., INT4)** after training, **without any fine-tuning**, while still preserving the original model's accuracy as much as possible.

Key Characteristics

- **Advanced PTQ method** using gradient-based optimization
- Designed **especially for Transformers and LLMs**
- Targets **INT4/INT8** quantization for model compression and **faster inference**
- Does **not require retraining** the model

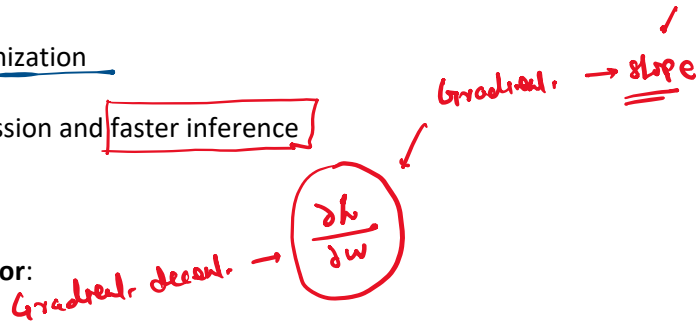
What Does "GPT" Mean in GPTQ?

Despite "GPT" in the name, **GPTQ** actually stands for:

Gradient Post-Training Quantization

Which means:

Use **gradients (second-order information: Gradient of gradient — Hessian matrix)** during the quantization process to **minimize quantization error — without retraining** the model.



7. What is GPTQ?

- **GPTQ (Gradient Post-Training Quantization)** is an **advanced Post-Training Quantization (PTQ)** method specifically designed for **Large Language Models (LLMs)** like **LLaMA, GPT, Mistral, Falcon**, etc.
- It quantizes model **weights to low precision (e.g., INT4)** after training, **without any fine-tuning**, while still preserving the original model's accuracy as much as possible.

Key Characteristics

- **Advanced PTQ method** using gradient-based optimization
- Designed **especially for Transformers based LLMs**
- Targets **INT4/INT8** quantization for model compression and faster inference
- Does **not require retraining** the model

What Does "GPT" Mean in GPTQ?

Despite "GPT" in the name, **GPTQ actually stands for:**

Gradient Post-Training Quantization

Which means:

Use **gradients (second-order information: Gradient of gradient — Hessian matrix)** during the quantization process to **minimize quantization error — without retraining** the model.

Concept	GPTQ Behavior
Type of Quantization	Post-Training Quantization (PTQ) — no model retraining required(Transformer-based LLMs (e.g., LLaMA, Falcon, GPT, BLOOM))
Quantized Precision	Typically INT4 , occasionally INT3 or INT2 for extreme compression
Target Components	Linear layers within Transformer blocks — including Q/K/V , FFN, etc.
Core Idea	Quantize weights layer-by-layer using second-order optimization to minimize output error , not just weight error
Accuracy	High — performance remains very close to the original FP16 model High (if properly calibrated and quantized)
Speed	Much faster than QAT — ideal for post-training model deployment Very fast inference (esp. with ExLlama, llama.cpp, etc.)
Calibration Data	Required: around 50–200 representative text samples for accurate quantization

Quantized GPTQ models can be saved in various formats depending on your deployment pipeline:

Format	Description
.safetensors	Secure & efficient format used by Hugging Face ecosystem (e.g., AutoGPTQ)
.gguf	Converted format for use with llama.cpp (fast inference on CPU/Mac)
.pt or custom	PyTorch checkpoint format or internal formats for specific libraries

GPTQ-for-LLaMA = original low-level implementation.

AutoGPTQ = easy Hugging Face integration.

ExLlama(V2) = fastest CUDA inference.

llama.cpp = CPU/Mac GGUF inference.

Transformers + bitsandbytes = efficient quantization & training support.

◇ GPTQ Tools Overview

AutoGPTQ

- Hugging Face-compatible wrapper around GPTQ.
- High-level API → makes loading and running GPTQ quantized models very easy.

GPTQ-for-LLaMA

- Original low-level repo by *Yao Fu et al.* implementing GPTQ from scratch.
- Base project from which other wrappers emerged.

ExLlama & ExLlamaV2

- Specialized fast inference backends for GPTQ quantized models (optimized for CUDA GPUs).
- ExLlamaV2 improves memory efficiency and speed further.

llama.cpp

- Runs models in **GGUF format** (including GPTQ-quantized) on CPU and Mac (Metal backend).
- Best choice for lightweight deployment.

Hugging Face Transformers + bitsandbytes

- Complements GPTQ workflows, mainly for **QLoRA / 4-bit & 8-bit quantization**.
- More for quantization-aware training and efficient loading, not direct GPTQ inference.

Typical PTQ:

- Post-Training Quantization (PTQ) usually just replaces full-precision weights with low-precision (e.g., INT4 or INT8) equivalents.
- It focuses on minimizing the **difference between original and quantized weights**.
- But this doesn't always ensure that the model's **final output** remains close to the original.

GPTQ is smarter:

1. **Tracks the error** that quantization introduces in each layer — **not just at the weight level**, but also how it affects the model's output.
2. **Uses second-order information** like gradients or **Hessian approximations** to understand the sensitivity of weights — i.e., which weights are more important for the model's performance.
3. GPTQ then selectively quantizes weights in a way that **minimizes the final output error**, not just weight-wise difference.

Simple Terms:

- **Standard PTQ** just compresses numbers using **min/max scaling**.
- **GPTQ** is smarter: it checks **how much quantizing a weight will affect model outputs**, and avoids high-impact quantization errors using the Hessian and **greedy + compensatory logic**.

GPTQ vs Standard PTQ: Key Differences

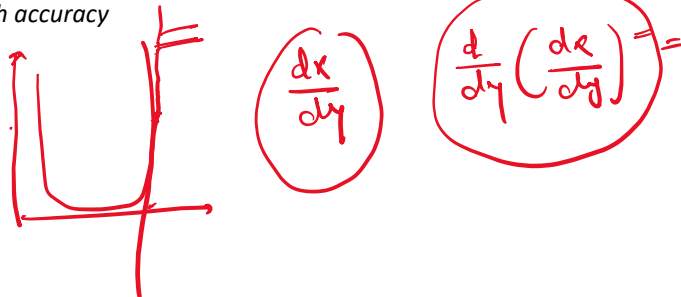
Verified Comparison: Standard PTQ vs GPTQ (Advanced PTQ)		
Aspect	Standard PTQ (Basic)	GPTQ (Advanced PTQ)
Goal	Quantize weights to INT8/INT4	Same goal, but minimizes output error more precisely
Formula	$x_q = \text{round}\left(\frac{x}{\text{scale}}\right) + \text{zero_point}$ Simple scalar rounding based on min/max	$\min_{\hat{W}} \ XW^T - X\hat{W}^T\ _F^2$ Matrix approximation to reduce output deviation
Level	Mostly <u>tensor-wise</u> or <u>channel-wise</u>	Always <u>row-wise</u> quantization (per weight vector)
Error Awareness	✗ No modeling of how quantization affects final output	✓ Uses Hessian (2nd-order gradients) to estimate the error introduced
Error Compensation	✗ No	✓ Yes — adjusts quantization to compensate future layer errors
Calibration Data	Optional (often skipped)	✓ Mandatory (~50–200 text samples) to compute statistical structure like $X^T X$
Math Behind	Basic linear scaling	Linear algebra: matrix decomposition + 2nd-order correction
LLM Use Case	✗ Rarely sufficient for Transformer ⁿ LMs	✓ Specifically designed for LLMs like LLaMA, GPT, Falcon, etc.

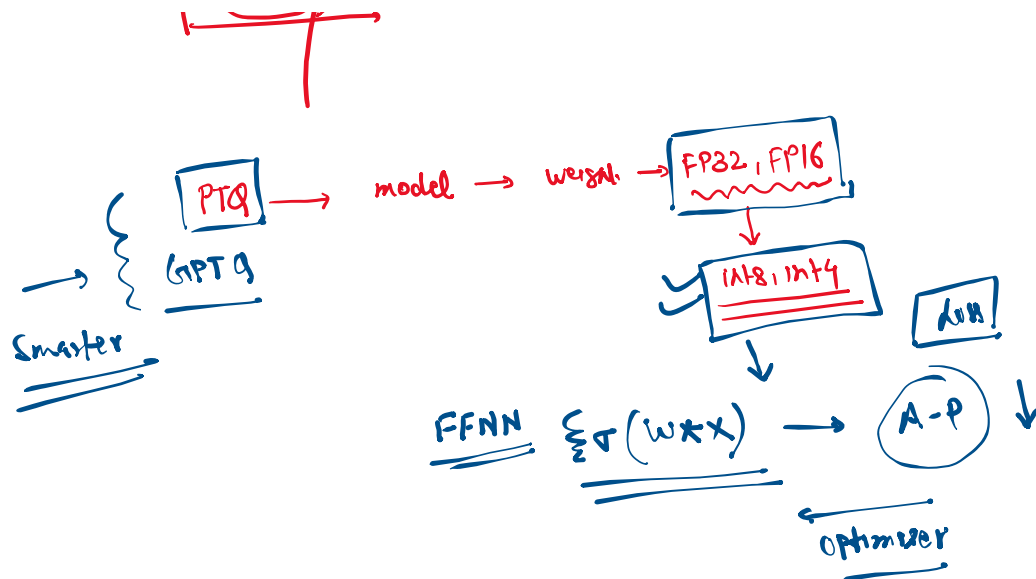
Matrix approximation = replacing a large, complex matrix with a simpler version that's "close enough."

- Methods: low-rank (SVD).
- Purpose: reduce memory + accelerate computation with minimal accuracy drop.

• Calibration data = a small representative dataset used during quantization to decide scales/ranges so that 4-bit/8-bit compressed weights/activations don't lose too much accuracy

- Hessian = **matrix of 2nd derivatives**.
- Captures curvature of a function.
- **First derivative** → slope.
- **Second derivative** → curvature.





$$PTQ = W \rightarrow w_q \rightarrow w_{dq}$$

$$= w_{dq} - w_q = \text{error}$$

$$\text{model} \rightarrow \text{OIP} \rightarrow \text{loss}$$

$$\text{GPT Q}$$

$$\text{PTQ} \text{ NN}$$

3x3		
1	4	7
2	5	8
3	6	9

Per tensor quantization

Per channel quantization

LLM

Observation:

Core Idea Behind GPTQ

GPTQ treats quantization as a **matrix approximation problem**:

You're replacing the original full-precision weight matrix W with a quantized version $\hat{W}$ (W-hat).

Given a large matrix (with high precision values), can we find a similar matrix with lower precision (or smaller size) that produces nearly the same result when used in computations

Original Matrix (W) **Quantized Matrix ($\hat{W}$)**

2.743 ————— 3 ✓
1.879 ————— 2 ✓
-0.356 ————— 0 ✓

Objective:

Minimize the output difference caused by using $\hat{W}$ instead of W , i.e.

$$W \cdot X \approx \hat{W} \cdot X$$

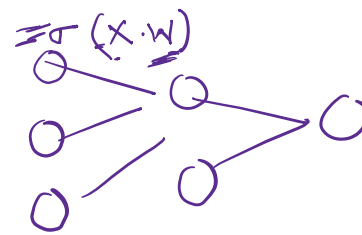
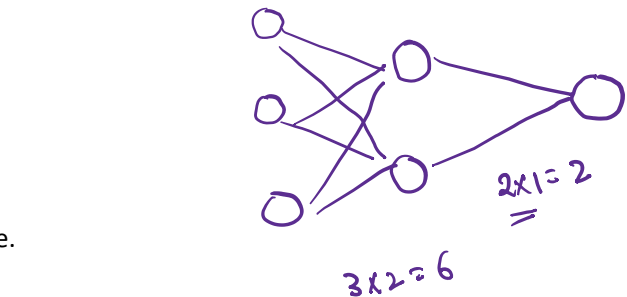
Where:

- W = Original weight matrix =
- $\hat{W}$ = Quantized weight matrix =
- X = Input activations =

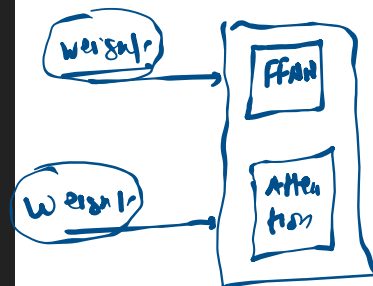
Original Weights → Quantized Weights → Output Compare

Step-by-Step Mathematics of GPTQ

Step	Formula
Linear Output	$Y = XW^T$
Loss Function	$\ XW^T - X\hat{W}^T\ _F^2$
Hessian (Row-wise)	$H = X^T X$
Quant Error Estimate	$(w - \hat{w})^2 \cdot H$
Error Compensation	$w' = w - \text{correction}$



Transformation



Summary of Flow:

1. Compute original output: $Y = XW^T$
2. Define loss to minimize: difference in output between W and $\hat{W}$
3. Compute Hessian: tells us how sensitive output is to changes in weights
4. Use that to quantize weights while minimizing impact on output
5. Apply error compensation to maintain global accuracy

Calibration Dataset

- GPTQ requires a small set of **real inputs (50–200 samples)** to simulate real inference data.

- This helps compute XXX and hence HHH, making quantization more accurate.

1. Model Layer Representation

- The output of the linear layer:

$$Y = XW^T$$

$$X [b \times n] \rightarrow W.T [n \times d] = Y [b \times d]$$

$$X = \begin{bmatrix} 1.0 & 2.0 & 3.0 \\ 0.5 & -1.0 & 2.0 \end{bmatrix} \quad [2 \times 3]$$

✓ Input

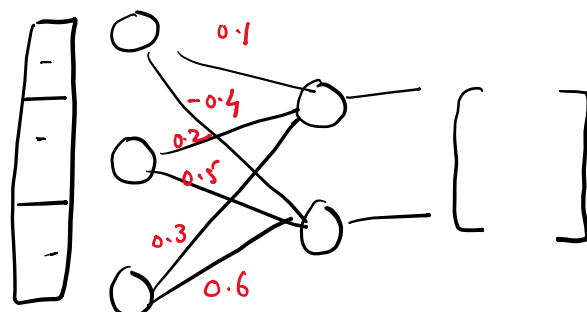
$$W^T = \begin{bmatrix} 0.1 & -0.4 \\ 0.2 & 0.5 \\ 0.3 & 0.6 \end{bmatrix} \quad [3 \times 2]$$

✓ weight

weight
quantize this
weights

$$Y = \begin{bmatrix} 1.0 & 2.0 & 3.0 \\ 0.5 & -1.0 & 2.0 \end{bmatrix} \cdot \begin{bmatrix} 0.1 & -0.4 \\ 0.2 & 0.5 \\ 0.3 & 0.6 \end{bmatrix}$$

XW^T



$$Y = \begin{bmatrix} 1.4 & 2.4 \\ 0.45 & 0.5 \end{bmatrix}$$

⇒ o/p

Explanation:

- This is the basic forward pass in a linear layer.
- You take input activations X and multiply them with the weight matrix W .
- The output Y is what the model predicts before quantization.

2. Quantization Goal

We want a quantized version $\hat{W}$ ($\hat{W}$) such that:

$$\text{Loss} = \|XW^T - X\hat{W}^T\|_F^2$$

Here:

- $\|\cdot\|_F^2$ is the Frobenius norm (sum of squared differences).
- The loss is minimized over rows of W , one at a time.

This is why GPTQ is row-wise quantization

$$= \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$

Explanation:

- You're comparing the original output (using real weights W) vs the quantized output (using $\hat{W}$).
- The Frobenius norm $\|\cdot\|_F^2$ just means sum of squared errors.
- Goal: Find $\hat{W}$ (quantized weights) that make this difference as small as possible.

- The Frobenius norm $\| \cdot \|_F^2$ just means sum of squared errors.
- Goal: Find $\hat{W}$ (quantized weights) that make this difference as small as possible.

But the question over here from where this quantized weights are coming?

Step What GPTQ Does

1. Takes one row of weights
2. Computes output using real input data
3. Tries all possible 4-bit values for each weight
4. Measures which rounding causes the least error
5. Picks smart quantized value
6. Compensates in next rows for any introduced error

GPTQ

quantized

make this worse

as bad as

small

4 bit range

-128, +127

3. Hessian Matrix Approximation

To preserve the model's output better, GPTQ incorporates second-order error correction using the Hessian matrix:

- Approximate local curvature of the loss landscape w.r.t. weights.
- Hessian for each row i of W :

$$H_i = X^T X$$

$$\frac{\partial L}{\partial w} \Rightarrow \left(\frac{\partial^2 L}{\partial w^2} \right) \rightarrow \frac{\partial}{\partial w} \left(\frac{\partial L}{\partial w} \right)$$

Gradient of Gradient

Real Input (X) $\rightarrow$ Compute Hessian ($X^T X$) $\rightarrow$ Get Sensitivity per Row $\rightarrow$ Do Safer Quantization

One-Line Summary

Hessian in GPTQ = A smart tool to identify which weights are risky to quantize, so GPTQ can do it safely without messing up output accuracy.

This is a covariance matrix of the inputs. It tells us which directions in W are more sensitive to changes.

Explanation:

This is a second-order matrix used for error sensitivity.

It tells us: if I make a small change in weight, how much does the output change?

We compute this for each row of weights (per output neuron), hence "row-wise".

Step What it does

1. Takes real input X
2. Computes $H = X^T X H = X^T X$
3. This gives a value telling how sensitive output is to each weight
4. GPTQ uses this to choose safer quantized values

Computation $\rightarrow$ loss $\rightarrow$ gradient

Hessian

$X^T X$

4. Quant Error Estimate

Formula:

$$(w - \hat{w})^2 \cdot H$$

Explanation:

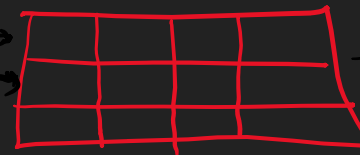
- Here, w is the real weight, $\hat{w}$ is the quantized weight.
- So, $(w - \hat{w})^2$ is the square of the quantization error.
- Multiply by H $\rightarrow$ this tells us how "important" this quant error is, i.e., how much it impacts final output.

- So, $(w - \hat{w})^2$ is the square of the quantization error.
- Multiply by $H \rightarrow$ this tells us how "important" this quant error is, i.e., how much it impacts final output.
- GPTQ uses this to decide which weights to quantize more carefully.

5. Error Compensation

Formula:

$$w' = w - (\text{correction})$$



Network

$$\Rightarrow (w - \hat{w})^2 \cdot H$$

$$\Rightarrow w' = w - \text{correction}$$

Explanation:

- Once a row (or block) is quantized, some error is introduced.
- GPTQ compensates future rows by slightly adjusting their weights (this is called Error Propagation Compensation).
- w' is the new corrected weight used in the next step.

Goal: Understand how GPTQ uses Hessian Matrix to do smart quantization

Step 1: Setup the Problem

Let's say we have a linear layer in a neural network:

$$Y = XW^T$$

Where:

- X = Input matrix (from real calibration data)
- W = Original full-precision weight matrix ← Quantize
- Y = Output of the layer

Quantization



transformer

Attention

NN

Suppose we have:

- Input size (features): $d = 2$
- Output size (neurons): $n = 1$ (i.e., 1 output neuron)
- Batch size: $b = 3$ (3 examples at once)

Define the matrices:

Input (X):

$$X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \quad (\text{shape: } 3 \times 2)$$

♦ Weight vector (W):

Since we have only 1 output neuron, weight matrix is 1×2 :

$$W = \begin{bmatrix} 0.5 & -1.0 \end{bmatrix} \quad (\text{shape: } 1 \times 2)$$

Since we have only 1 output neuron, weight matrix is 1×2 :

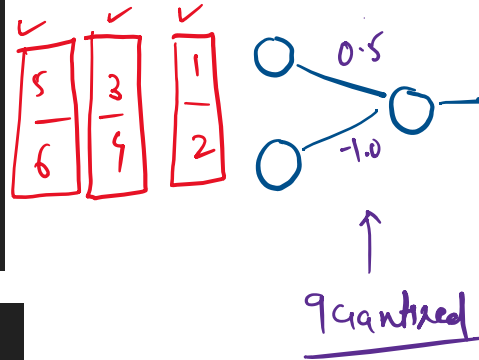
$$W = \begin{bmatrix} 0.5 & -1.0 \end{bmatrix} \quad (\text{shape: } 1 \times 2)$$

Step 2: Compute Output

We compute:

output

$$Y = XW^T = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \cdot \begin{bmatrix} 0.5 \\ -1.0 \end{bmatrix} = \begin{bmatrix} 1 \times 0.5 + 2 \times (-1.0) = -1.5 \\ 3 \times 0.5 + 4 \times (-1.0) = -2.5 \\ 5 \times 0.5 + 6 \times (-1.0) = -3.5 \end{bmatrix}$$
$$Y = \begin{bmatrix} -1.5 \\ -2.5 \\ -3.5 \end{bmatrix}$$



Step 3: Now Suppose You Quantize W

Say:

$$\hat{W} = \begin{bmatrix} 0.5 & -0.5 \end{bmatrix}$$

Quantized second weight is off from $-1.0 \rightarrow$ to -0.5

♦ New Output:

$$\hat{Y} = X \cdot \hat{W}^T = \begin{bmatrix} 1 \times 0.5 + 2 \times (-0.5) = -0.5 \\ 3 \times 0.5 + 4 \times (-0.5) = -0.5 \\ 5 \times 0.5 + 6 \times (-0.5) = -0.5 \end{bmatrix} = \begin{bmatrix} -0.5 \\ -0.5 \\ -0.5 \end{bmatrix}$$

Observation:

Error:

$$\text{Error} = Y - \hat{Y} = \begin{bmatrix} -1.5 \\ -2.5 \\ -3.5 \end{bmatrix} - \begin{bmatrix} -0.5 \\ -0.5 \\ -0.5 \end{bmatrix} = \begin{bmatrix} -1.0 \\ -2.0 \\ -3.0 \end{bmatrix}$$

Error is large — quantization harmed output badly.

Step 4: Hessian Matrix Calculation

GPTQ uses:

$$H = X^T X \quad (\text{shape: } 2 \times 2)$$

Let's calculate:

$$X^T = \begin{bmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{bmatrix}$$

Now:

$$H = X^T X = \begin{bmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{bmatrix} \cdot \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} =$$

Matrix multiplication:

$$H = \begin{bmatrix} (1 \times 1 + 3 \times 3 + 5 \times 5) & (1 \times 2 + 3 \times 4 + 5 \times 6) \\ (2 \times 1 + 4 \times 3 + 6 \times 5) & (2 \times 2 + 4 \times 4 + 6 \times 6) \end{bmatrix} = \begin{bmatrix} 35 & 44 \\ 44 & 56 \end{bmatrix}$$

Step 5: Use Hessian to Judge Sensitivity

Now GPTQ computes error due to quantizing a weight:

$$(w - \hat{w})^2 \cdot H$$

$$(w - \hat{w})^2 \cdot H$$

Let's say you're changing:

- $w_2 = -1.0 \rightarrow \hat{w}_2 = -0.5$

So:

$$(w - \hat{w})^2 = (-1.0 + 0.5)^2 = 0.25$$

Now error due to this change along 2nd dimension:

$$\text{Impact} = 0.25 \times H_{2,2} = 0.25 \times 56 = 14$$

Big impact = this weight is **sensitive**

So GPTQ **won't pick** -0.5 for quantization of that weight

Step Action

- 1 Take real calibration input X
- 2 Compute output using original weights W
- 3 Try quantized weights $\hat{W}$, observe new output
- 4 Calculate Hessian: $H = X^t X$
- 5 For each weight: compute error impact as $(w - \hat{w})^2 \times H_{i,i}$
- 6 Choose quantized weight that gives **minimum error**
- 7 Repeat for all rows in W (i.e., per neuron)

AWQ: Activation-aware Weight Quantization

AWQ is a **Post-Training Quantization (PTQ)** method like GPTQ, but it improves output accuracy by also considering the **activation distribution** during quantization.

Intuition Behind AWQ

- Instead of blindly quantizing weights, AWQ checks how each weight affects the output, after going through activation functions (like GELU, ReLU, etc).
- then prunes and rescales weights so that they cause minimum distortion in output — especially on important tokens during inference.

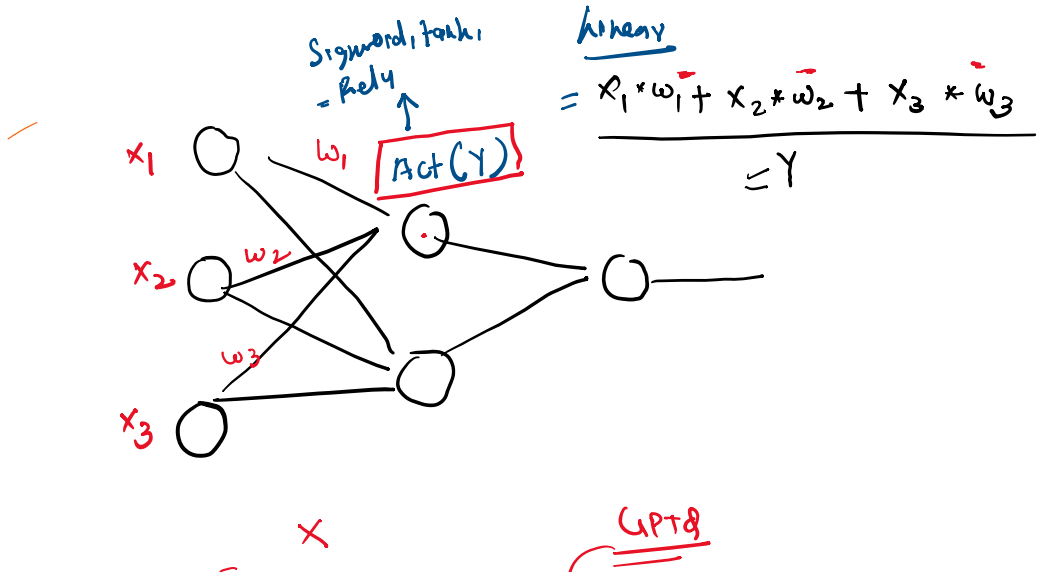
Feature	GPTQ	AWQ	Post-training quantization	[GPTQ] → AWQ
Error Modeling	Weight-only error	Weight + Activation error		
Uses Hessian	Yes	No (simpler & faster)		Memran → Gradient → Gradient.
Calibration Needed	Yes (50–200 samples)	Yes (same)		
Precision	INT4 (mainly)	INT4 (mainly)		
Speed	Fast	Even faster (no second-order calc)		
Accuracy	High	Very High (better than GPTQ in some LLMs)		
Quant Target	Weights	Weights + Activation impact		
Use Case	LLaMA, GPT	LLaMA, Mistral, ChatGLM, etc.		

Real-Life Analogy

- GPTQ** = Measures how much *raw ingredients* changed
- AWQ** = Measures how the *final dish tastes* after cooking (activation)

Why is this Better Than GPTQ in Some Cases?

GPTQ	AWQ
Focus on pre-activation error	Focus on post-activation (real output) error
Uses second-order gradients (Hessian)	No gradients – lighter & faster
Slower on large models	Faster, no Hessian
Good for precision	Better for both precision & speed (especially in inference-only scenarios)



$$\begin{array}{c} X \\ \left[\begin{array}{ccc} 1 & 2 & 3 \\ 4 & 5 & 6 \end{array} \right] \end{array} \Rightarrow \begin{array}{c} \text{GPTq} \\ \boxed{XW^T} \end{array} \Rightarrow Y$$

$$\begin{array}{c} W \\ \left[\begin{array}{ccc} 0.53 & 0.45 & 0.36 \\ 0.32 & 0.551 & 0.565 \end{array} \right] \end{array} \xleftarrow{\text{Avg}} \underline{\underline{\text{Act}(XW^T) = Y}}$$

AWQ Mathematics – Breakdown

Problem Setup:

In any Transformer-based LLM:

- X = Input activation matrix (tokens $\rightarrow$ hidden states)
- W = Weight matrix of a linear layer (Q, K, V, FFN etc.)
- f = Non-linear activation (GELU, ReLU etc.)
- $\hat{W}$ = Quantized weights

The output of a layer is:

$$Y = f(XW)$$

When quantized:

$$\hat{Y} = f(X\hat{W})$$

Goal of AWQ is to minimize output error:

$$\min_{\hat{W}} \|f(XW) - f(X\hat{W})\|_F^2$$

Ye GPTQ se different hai. GPTQ minimize karta hai:

$$\|XW - X\hat{W}\|_F^2 \quad (\text{before activation})$$

But AWQ includes non-linearity, so it's more realistic and better for preserving accuracy in LLMs.

Why is this Better Than GPTQ in Some Cases?

GPTQ

Focus on pre-activation error

Uses second-order gradients (Hessian)

Slower on large models

AWQ

Focus on post-activation (real output) error

No gradients – lighter & faster

Faster, no Hessian

QUANTIZATION-AWARE TRAINING (QAT)

Summary:

Simulates quantization **during training** so model learns how to handle low-precision.

How it works:

- Insert **fake quant/dequant ops** during training
- Backprop still uses float32, but forward uses simulated int8
- After training, you convert to real int8 model

1. Decide what to quantize (Linear layers).
2. Insert fake quantization (prepare_qat).
3. Train or Fine-tune with representative data.
4. Convert to real INT8 model (convert).
5. Evaluate, possibly skip sensitive layers.

Choose what to quantize

You don't quantize *everything*, because:

- **Embeddings** → usually kept in FP32 or weight-only quantized.
- **LayerNorms** → kept in FP32.
- **Final lm_head** → often skipped (kept FP32).

Focus is usually on **Linear layers** in Attention and MLP blocks:

- q_proj, k_proj, v_proj, o_proj
- up_proj, gate_proj, down_proj

Why QAT expensive for LLMs?

1. Model Size

- MLPs or CNNs → only a few million parameters.
- LLMs → **billions of parameters** (7B, 13B, 70B).
- Applying fake quantization + backpropagation on every parameter = **huge compute + memory cost**.

2. Fake Quantization in Every Forward Pass

- In QAT, every forward pass applies FakeQuantize: rounding + clamping.
- This adds extra computation overhead.
- Large models + long sequence lengths → cost multiplies.

3. Backpropagation Cost

- Even with fake quantization, **gradients are computed in FP32**.
- That means no memory savings during training.
- You still need the **full VRAM footprint** of the original model.

4. Retraining Requirement

- In PTQ, the model is quantized without retraining.
- In QAT, you must **retrain / fine-tune** the model on representative data.
- Fine-tuning a billion-parameter LLM = requires multi-GPU clusters or TPU pods.

5. Resource Requirement

- Example: Proper QAT on LLaMA-7B → at least **4-8 A100 80GB GPUs**.
- QAT on LLaMA-65B → dozens of GPUs (industrial scale).
- That's why big labs (Meta, Microsoft, OpenAI) can afford it, but it's too expensive for individuals.

QAT →

Simulating

Lower Precision

Weights

QAT

Projection

matrix

LLM

training → resource intensive task

-128 to 127

QAT

Computation GPU

{ slower }

27.01

27

10-255

PTQ → retraining

QAT → retrain / training

- Example: Proper QAT on LLaMA-7B → at least 4-8 A100 80GB GPUs.
- QAT on LLaMA-65B → dozens of GPUs (industrial scale).
- That's why big labs (Meta, Microsoft, OpenAI) can afford it, but it's too expensive for individuals.

Alternatives

- PTQ (GPTQ, AWQ) → just calibration data + a few forward passes, **cheap**.
- LoRA + QAT (Partial QAT) → apply QAT only on Linear/Adapter layers, not the whole model.
- Hybrid approach → keep embeddings in FP32, quantize Linear/Attention layers to INT8/4-bit.

Partial QAT

QAT → quantize aware training

Scratch → LLM ← challenging

while we training the LLM ← QAT

After the training ⇒ (FT) ← QAT

Distillbert

→ Llama / mistral / Deepseek

HF model → **quantize (Q4/Q8)** → **.gguf** → **load in llama.cpp or GGML engine**

LLM *.bin* *quantized* *GGML* *load* *LLM or QLLM* *CPU Apple Silicon embedded smaller confs*

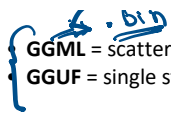
- **GGML = Georgi Gerganov's Machine Learning framework.**
- It is **not** "GPT-Generated Model Language" (common myth online).
- It is a **C-based tensor library**.
- The **core GGML library's** main job is to efficiently **run quantized operations** on **CPU** or **Apple Silicon**.
- **GGML = An inference engine (library) that runs quantized models.**
- **GGML .bin format = The old model file format.**
- **GGML was using its own quantization formats** (Q4_0, Q4_K, Q5_0, Q8_0).
- **GGUF = A new standardized file format (not an engine), which stores models in a single file along with all metadata.**
- **GGML itself does not perform quantization.**
- The **GGML ecosystem (tools from llama.cpp)** provides **quantization** → converting models into .bin or .gguf quantized formats. (using scripts (convert.py, convert-llama-gguf.py))
- **Quantization:** Supports INT8, INT4, and custom formats like **Q4_0**, **Q4_K**, etc.
 - **Q4_0 (4-bit):** Basic compression, smallest size but lower accuracy.
 - **Q4_K (4-bit):** Smarter quantization (similar to AWQ, keeps important weights more accurate).
 - **Q5_0 (5-bit):** Balanced trade-off between speed and accuracy.
 - **Q8_0 (8-bit):** Near-FP16 quality with good performance.
- **Not GPTQ/AWQ/QAT.** → *GGML llama.cpp*
- **Use case:**
 - Runs **LLaMA, Whisper, GPT models** on **laptops/mobile devices** with **CPU**.
 - Originally worked with **.bin model files** (quantized via **GGML format**).
 - **Model Types:** Optimized for **decoder-only LLMs** (e.g., LLaMA, Falcon).
- **GGML can run standalone**, but it's **mostly used via llama.cpp**, which provides:
 - Text generation
 - Prompt processing
 - Streaming outputs
 - Partial GPU offloading
 - Quantized inference on CPU/Mac
- **GGUF (GGML Unified Format / Gerganov's General Unified Format):**
 - A **standardized binary file format** for **storing & deploying quantized LLMs**.
 - Primarily used in the **llama.cpp ecosystem**.
 - Successor to the old **GGML .bin format**.
 - **Naming:**
 - Official: **GPT-Generated Unified Format**.
 - Community also uses: **Gerganov's General Unified Format** (since Georgi Gerganov made it).
 - **Design:**
 - Created by the **llama.cpp community**.
 - A **self-describing format** → contains **both weights + metadata**.
 - Metadata includes: **tokenizer, vocab, RoPE params, special tokens**, etc.
 - Makes deployment easier → **tools don't need manual flags**.
 - **Use Case:**
 - Store and run LLMs (e.g., LLaMA) in a **portable way**.
 - Hugging Face + llama.cpp now fully support GGUF.

Feature	GGML (Old)	GGUF (New)
Format type	Old quantized format	New unified format
File extension	.bin files	.gguf files
Tokenizer	Not included inside file	Tokenizer embedded in file
Versioning	No versioning	Versioned & standardized
Status	Still works (legacy support)	Recommended to migrate (future-proof)

Problem in GGML .bin → **GGUF Solution** ← *llama.cpp*

- Separate files for tokenizer & metadata → Single file with **everything packed**
- Not versioned properly → **Formal versioning + backward compatibility**
- Hard to inspect → **Human-readable metadata** (via gguf-tool)
- Lacked standard tokenizer info → **Tokenizer, vocab, and pre-tokenizer embedded**
- Limited support for new quantization types → **Native support** for INT4, Q8_0, Q6_K, etc.

GGML = scattered files, no proper versioning, inspection tough.
GGUF = single standardized format, packed metadata, better quantization support, easy tooling.



GGML = scattered files, no proper versioning, inspection tough.

GGUF = single standardized format, packed metadata, better quantization support, easy tooling.

initial days $\Rightarrow$ (GGML) $\Rightarrow$ C based library $\rightarrow$ (quantization) explicitly
 $=$ was not tends to perform

HF $\rightarrow$ (GGML)
dem $\rightarrow$ specific quantized format
qem $\rightarrow$

Llama.CPP $\Rightarrow$ Converts normal model to quantize model